

Why Is My Food Late?

FlashEats applied challenge · Understanding client data & source systems

Client request: "Our customers are angry about late deliveries. Build an AI system that predicts delays and automatically fixes the problem."

Constraint: no architecture and no model. Only the information behind the promise: what we need to know, where it lives, who owns it, and whether we can trust it.

1 Information needs: what must we know to explain a delay?

Start from the business question, not the systems. "Why is this order late?" breaks into seven questions, each of which needs specific information.

Business question	Information need	Required field / event
Is the order actually late?	What was promised, and when it arrived	promised_eta, delivered_at, final status
Was food preparation late?	When the restaurant accepted, started and finished	accepted_at, prep_started_at, ready_at
Was driver assignment late?	When a driver was assigned, and any reassignment	driver_assigned_at, reassigned_at, original vs current driver
Did the driver reach the restaurant late?	When the driver physically arrived	driver_arrived_at_restaurant (does not exist)
Was pickup delayed?	Gap between food ready, driver arrival and pickup	pickup_confirmed_at vs ready_at
Is traffic causing the delay?	Expected vs actual route time	Route duration, congestion, GPS trace
Is the displayed ETA itself wrong or stale?	What the customer saw, and when it changed	ETA history (value + timestamp), app status

2 Source map: which source should provide each piece of information?

Information	Source	Owner	Format	Freshness	Can I trust it?
Promised ETA, lifecycle, delivered_at, final status	Order platform (DB); orders.csv is only an export	Commerce / platform team	SQL	Live DB; CSV lags 30 min	HIGH for the DB. The CSV is a snapshot, so use it for history, not live status
Delivery ETA, pickup ETA, assignment, reassignment	Dispatch service	Dispatch / logistics engineering	API	Live, recalculated	HIGH for what dispatch <i>decided</i> . The ETA changes, so its history matters
Restaurant accepted / preparing / ready	Restaurant API	Restaurant partner ops	API	Only when staff update it	LOW Correct owner, but the status is often updated manually and late
Driver location, driver status	Driver location API	Driver app team	API (GPS)	Stale by 2-5 min	MEDIUM Owner is clear; accuracy and freshness are not
Route time, congestion	Maps / traffic API	External vendor	API	Live	MEDIUM An <i>input</i> to the ETA, not the business ETA
What the customer saw	Customer app	Consumer app team	Client logs	Displays dispatch data	MEDIUM A display layer; can show a cached value

Customer's experience of the delay	Support tickets	Support lead	Free text	After the fact	LOW as an event log. Useful context: subjective and unstructured
------------------------------------	-----------------	--------------	-----------	----------------	--

Structured (SQL/CSV): order platform. Semi-structured (JSON/API): dispatch, restaurant, driver, maps. Unstructured: tickets. Relevant does not mean authoritative: tickets and the app are relevant but authoritative for nothing operational.

3 Systems of record, with the reasoning written down

Fact	System of record	Why (ownership · replication · freshness · semantics)
Delivery ETA	Dispatch service	Dispatch <i>creates</i> the platform ETA. The app only displays it, and Maps is an input to it. The order platform holds the <i>original promise</i> . These are two different facts (promised vs current ETA), and "late" must say which one it's measured against.
Driver assignment	Dispatch service	Assignment and reassignment originate here. The driver app and order platform only receive copies.
Restaurant readiness	Restaurant system LOW TRUST	It officially owns the state, but staff update it by hand, so it's the correct source and an unreliable one. The restaurant employee's claim ("ready for 12 minutes") is human context, not a record.
Delivery completion	Order platform	It owns the order lifecycle and final status. To confirm: which system actually raises the completion event (driver app tap → order platform). If the driver app is the origin, the order platform is a near-real-time copy. The orders.csv export is never the source of record.

The rule applied: a source of record is the system that *creates and changes* the business object, not the one updated most recently. If the app refreshed a minute ago, that doesn't make it the truth. Freshness alone is not authority.

! Applying it to the live incident: order #FE-48291 at 7:31 PM

Source	Says	Read as
Customer app	ETA 7:42 · "Driver picking up"	Stale display. Dispatch already says 7:51
Dispatch	Pickup 7:34 · Delivery 7:51	Current platform ETA (SoR)
Restaurant API	PREPARING · updated 7:21	10 minutes stale; contradicts staff
Restaurant staff	"Ready for 12 min" (≈7:19)	Human claim, unverified
Driver API	1.8 km from restaurant · GPS 7:29	Driver has <i>not</i> arrived
Maps	Traffic +11 min	Input, not a cause by itself
Ticket	"App keeps changing the ETA"	Symptom: ETA instability

Most likely story: the food has probably been ready since about 7:19, but the restaurant never updated its status. The driver is still 1.8 km away (and that GPS reading may be 2–5 minutes old), so the delay looks **driver-side, before pickup**. Traffic adds to it on the way. Separately, the customer is seeing an **outdated ETA** (7:42 vs dispatch's 7:51), which is why "10 minutes away" hasn't changed for 20 minutes.

Can we answer confidently? No.

- "Food ready" rests on a verbal claim; the system says otherwise.
- Without an arrival event we can't say whether the driver was assigned late, took a slow route, or was reassigned.
- The GPS reading is approximate and possibly stale.

Two problems are mixed together: an **operational delay** (who is slow) and an **information failure** (what the customer is shown). They need different fixes.

4 Data risks

Risk type	Where	Why it matters	Mitigation
STALE	GPS lags 2–5 min; orders.csv lags 30 min; the app shows a cached ETA	Live decisions made on old data. The export can't answer "where is my order now"	Read the live sources for operations; check freshness against each use case
MANUAL	Restaurant status updated by staff	"Preparing" may mean "we forgot to press ready". Blaming restaurants on this is unfair	Treat as low trust; instrument a <code>food_ready</code> event (e.g. tied to the order being bagged)
CONFLICTING	App ETA vs dispatch ETA; restaurant API vs staff	Different systems give different answers at the same moment	Declare a source of record per fact (section 3); never "take the latest"
INCOMPLETE	No driver-arrival event; ETA changes overwritten rather than kept	Delay can't be attributed; ETA instability can't be measured	Add the arrival event; store ETA history
AMBIGUOUS	"Late" (vs original or current ETA? any delay or >N min?); "picking up"	Two teams can compute different late rates from the same data	Named KPI owner and a written definition before any metric is published
SUBJECTIVE	Support tickets, free text	Shows experience, not events; only some angry customers write in	Use as context and to prioritise, not as a delay label

5 Missing information: what stops us explaining the delay confidently?

- **driver_arrived_at_restaurant is not stored.** The platform records assignment, approximate GPS, pickup and delivery, but not arrival. Without it, the time between order and pickup can't be split into *restaurant was slow* and *driver was late*. **This is the single most important gap.**
- **A reliable food_ready timestamp.** The field exists, but because it's manual, the time it records isn't when the food was actually ready.
- **ETA history.** We see the current ETA, not the sequence of ETAs the customer saw, so "the ETA keeps changing" can't be measured.
- **An agreed definition of "late"** and a named owner for it.
- **A reliable join** between tickets and orders (tickets are free text; some may not carry an order ID).

Is this a data-quality problem or a workflow problem? Mostly **workflow and instrumentation**. Inferring arrival from GPS geofencing would be *inferred*, not observed, and 2–5 minute stale pings make it weak. The better fix is to add an "Arrived" tap in the driver app, or automatic instrumentation, and until then narrow the MVP scope to what the data can support.

6 Revised problem: is this really an "AI prediction problem"?

Not yet. A predictor needs a label (was this order late, and why?), and the system can't currently produce one reliably: "late" isn't defined, and the "why" depends on an event that isn't recorded.

FlashEats cannot reliably attribute delivery delays, because key operational events, particularly driver arrival and restaurant readiness, are missing or recorded inconsistently, and "late" has no agreed definition. Customers are also shown an ETA that goes stale and keeps changing, which turns a delay into a trust problem.

#	Before building an AI delay predictor	Why first
1	Define "late" and name its owner	No label, no metric, no model

2	Instrument driver arrival and restaurant readiness	Makes attribution possible
3	Declare canonical ownership for each delivery state	Resolves conflicting sources
4	Validate timestamp freshness and semantics; store ETA history	Makes the data safe to act on
5	Only then evaluate prediction	AI can't reliably predict a label the system itself can't define

✓ Required-fields list: the minimum the first version needs

MVP scope: explain, per order, which stage of the lifecycle made it late. Start from the decision and work backwards, rather than ingesting everything we have access to.

Field	Source of record	Needed for
order_id, created_at, promised_eta	Order platform	Identity; the promise being measured against
delivered_at, final_status	Order platform	Whether it was late; excluding cancellations
driver_assigned_at, reassigned_at	Dispatch	Assignment delay
current_delivery_eta (with history)	Dispatch	ETA drift / what the customer was told
ready_at (flagged low trust)	Restaurant system	Preparation delay
pickup_confirmed_at	Order platform / driver app	Pickup delay; start of transit
driver_arrived_at_restaurant	To be instrumented	Splitting pre-pickup delay

Deliberately excluded for now: full GPS traces, customer profiles, raw support text, and a live traffic feed. They may become necessary after validation, but having access to them isn't a reason to collect them.

? Debrief

Question	Answer
Which source did you trust for the ETA, and why?	Dispatch. It creates and recalculates the ETA; the app only displays it and Maps only feeds it.
Which source worried you most?	Restaurant status. It's the official owner of readiness but is updated by hand, and in the live incident it contradicted the staff.
What information was missing?	Driver arrival at the restaurant, and a history of ETAs shown to the customer.
Would you build the AI delay predictor today?	No. Fix instrumentation and the definition of "late" first. The first win is probably communicating a stable, honest ETA.
What should we fix first?	Add the driver "Arrived" event and agree who owns the definition of "late".